VOL. 47

Hugging Face

Using Transformers library for NLP and beyond

Source	huggingface.co/docs
Generated	2026-10-11
Level	Intermediate
Estimated study time	25 hours
Language	English
Volume	Vol. 47 - Hugging Face

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume . . . 3

Chapter 1 - The Hugging Face ecosystem . . . 3

1.1 Summary . . . 3

1.2 Concepts . . . 3

1.3 Examples . . . 3

Example 1 - Using a pre-trained model . . . 3

1.4 Mini test . . . 4

Chapter 2 - Tokenization and model loading . . . 4

2.1 Summary . . . 4

2.2 Key concepts . . . 4

Chapter 3 - Pipelines for common tasks . . . 5

3.1 Summary . . . 5

3.2 Examples . . . 5

3.3 Mini test . . . 5

Chapter 4 - Fine-tuning . . . 5

4.1 Summary . . . 5

4.2 Key concepts . . . 6

Final test . . . 6

5.1 Final test answers . . . 6

Sources . . . 6

How to use this volume

Prerequisites. Python programming, basic machine learning concepts. Familiarity with PyTorch or TensorFlow is helpful.

Estimated study time. About 25 hours.

About the source. The syllabus follows Hugging Face's official documentation. All explanations, examples and exercises are original.

Chapter 1 - The Hugging Face ecosystem

1.1 Summary

- Hugging Face is a company and platform for NLP and ML.
- The Hub (huggingface.co/models) hosts pre-trained models.
- The Transformers library provides model classes.
- The Datasets library provides dataset loading and processing.
- The Tokenizers library provides fast tokenisation.
- Spaces hosts web apps (Streamlit/Gradio) for demos.

1.2 Concepts

Component	Purpose
`transformers`	Pre-trained models, training tools
`datasets`	Download and prepare datasets
`tokenizers`	Fast, efficient tokenisation
`hub`	Model and dataset repository
`spaces`	Deploy apps
`evaluate`	Metrics for model evaluation

1.3 Examples

Example 1 - Using a pre-trained model

Python

```python
# ch01_pretrained.py
from transformers import AutoTokenizer, AutoModel

model_name = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)

text = "Hello, how are you today?"
tokens = tokenizer(text, return_tensors="pt")
print(f"Input tokens: {tokens['input_ids']}")

# Get model outputs
with torch.no_grad():  # or model.eval()
    outputs = model(**tokens)

print(f"Output shape: {outputs.last_hidden_state.shape}")
print(f"Token hidden states: [batch_size, sequence_length, hidden_size]")
```

1.4 Mini test

1. What is the Hugging Face Hub? 2. What does AutoTokenizer do? 3. What are Spaces?

Answers: 1) A repository of pre-trained models and datasets. 2) Automatically loads the correct tokenizer. 3) Hosted web apps for demos.

Chapter 2 - Tokenization and model loading

(Same structure as Vol 44 Chapter 2 - abbreviated here)

2.1 Summary

- Tokenizers convert text to integer IDs.
- Different models use different tokenizer types.
- The tokenizer's `encode` method returns token IDs.
- `decode` converts IDs back to text.
- Special tokens: [CLS], [SEP], [UNK], [PAD].

2.2 Key concepts

Tokenizer type	Used by	Tokenization method
BertTokenizer	BERT	WordPiece
GPT2Tokenizer	GPT-2	BPE
XLNetTokenizer	XLNet	SentencePiece

Chapter 3 - Pipelines for common tasks

3.1 Summary

- The `pipeline` API abstracts away model loading and preprocessing.
- Available tasks: text-classification, token-classification, question-answering, fill-mask, summarization, translation, text-generation, feature-extraction, zero-shot-classification.
- Custom models can also be used with pipelines.

3.2 Examples

Python

```python
# ch03_pipelines.py
from transformers import pipeline

# Text classification (sentiment)
classifier = pipeline("sentiment-analysis")
print(classifier("I love Hugging Face!"))

# Question answering
qa = pipeline("question-answering")
answer = qa(
    question="Where do I live?",
    context="My name is Anna and I live in Berlin."
)
print(answer)

# Summarization
summarizer = pipeline("summarization")
summary = summarizer("Long article text here...")
```

3.3 Mini test

1. What does the fill-mask pipeline do? 2. What is the purpose of a pipeline? 3. How many tasks are available?

Answers: 1) Predicts masked tokens. 2) Simplifies model usage with minimal code. 3) About 10 major tasks.

Chapter 4 - Fine-tuning

4.1 Summary

- Fine-tuning adapts a pre-trained model to a specific task.
- Use the Trainer API or custom training loops.
- Save and upload your fine-tuned model to the Hub.

4.2 Key concepts

Step	Description
1	Load dataset (from Hub or local)
2	Tokenize
3	Create DataLoaders
4	Define training arguments
5	Train with Trainer
6	Evaluate and save

Final test

1. What is the purpose of the Hugging Face Hub?
2. What does `AutoModel.from_pretrained()` do?
3. What is a pipeline in Transformers?
4. What does fine-tuning do?
5. What are special tokens used for?

5.1 Final test answers

1. Host and share pre-trained models and datasets.
2. Loads a pre-trained model architecture.
3. A high-level API for common NLP tasks.
4. Adapts a pre-trained model to a specific task.
5. Mark sentence boundaries, padding, and unknown tokens.

Sources

- https://huggingface.co/docs
- https://huggingface.co/docs/transformers/main_classes/model
- https://huggingface.co/docs/datasets
- https://huggingface.co/docs/tokenizers